

Spring Boot a Nivel Senior: Guía Práctica e Internals

Apuntes de trabajo

19 de octubre de 2025

Índice

1 Introducción

Estos apuntes cubren Spring Boot 3.x (Spring Framework 6), Java 17+ y el stack moderno: auto-configuración, observabilidad (Micrometer + OpenTelemetry), AOT y native image, seguridad, datos, reactividad y despliegue. Incluye **internals** para comprender cómo funciona por debajo y tomar decisiones de arquitectura con criterio.

Consejo Senior

Mentalidad senior: simplicidad, límites claros, contratos estables (interfaces/DTOs), observabilidad desde el día uno, automatización y *shift-left testing*.

2 Glosario ultra básico (explicado simple)

- **Arrancar (bootstrap)**: iniciar la app. En Java: método `main` que llama a `SpringApplication.run(...)`.
- **Spring Boot**: capas y utilidades que configuran Spring “solos” por convención (*starters*, autoconfiguración).
- **Bean**: objeto gestionado por Spring (vida, dependencias). Se crea con `@Component`, `@Service`, `@Repository` o `@Bean`.
- **IoC/DI**: Inversión de Control/ Inyección de Dependencias. Spring crea e inyecta los beans donde se necesitan.
- **Controller/RestController**: clase que atiende peticiones HTTP y devuelve respuestas (JSON).
- **Propiedades**: configuración externa (`application.yml`). Puedes leerlas con `@Value` o `@ConfigurationProperties`.
- **JPA**: estándar para mapear objetos a tablas (ORM). Spring Data JPA te da repositorios listos.
- **Transacción**: bloque de trabajo todo-o-nada en BD, controlado con `@Transactional`.
- **Actuator**: endpoints de salud/métricas/info para operar la app.
- **WebFlux/Reactor**: modelo reactivo (no bloqueante) para alto número de conexiones.
- **AOP**: “envolver” métodos con lógica transversal (métricas, seguridad, transacciones).

3 Fundamentos prácticos: de cero a tu primera API

Objetivo: entender *qué es cada cosa y para qué sirve* con un ejemplo mínimo.

3.1 Cómo arranca una app

```
1 // src/main/java/com/example/App.java
2 package com.example;
3 import org.springframework.boot.autoconfigure.SpringBootApplication;
4 import org.springframework.boot.SpringApplication;
5
6 @SpringBootApplication // activa escaneo de componentes y autoconfiguración
7 public class App {
8     public static void main(String[] args) { SpringApplication.run(App.class,
9         args); }
10 }
```

Listing 1: Aplicación mínima

Qué pasa: Spring busca clases anotadas (beans), crea el “contenedor” (IoC) y las conecta.

3.2 Tu primer endpoint REST

```
1 // src/main/java/com/example/web/HelloController.java
2 package com.example.web;
3 import org.springframework.web.bind.annotation.*;
4
5 @RestController
6 class HelloController {
7     @GetMapping("/hello") String hello(@RequestParam(defaultValue="Mundo")
8         String name) {
9         return "Hola, " + name;
10    }
11 }
```

Listing 2: Hola mundo HTTP

Qué pasa: `@RestController` expone métodos HTTP. `@GetMapping` atiende GET `/hello`.

3.3 Servicios e inyección de dependencias

```
1 // src/main/java/com/example/core/GreeterService.java
2 package com.example.core;
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class GreeterService { public String greet(String name){ return "Hola,
7     " + name; } }
8
9 // src/main/java/com/example/web/HelloController.java
10 @RestController
11 class HelloController {
12     private final GreeterService greeter;
13     HelloController(GreeterService greeter) { this.greeter = greeter; } //
14     // inyección por constructor
15     @GetMapping("/hello") String hello(@RequestParam(defaultValue="Mundo")
16         String name) {
17         return greeter.greet(name);
18     }
19 }
```

Listing 3: Separar lógica en un servicio

Idea: el controlador solo orquesta; la lógica vive en servicios.

3.4 Propiedades y configuración

```
1 spring.application.name: demo
2 app.title: "Demo App"
```

Listing 4: application.yml

```
1 // src/main/java/com/example/config/AppProps.java
2 @ConfigurationProperties(prefix="app")
3 public record AppProps(String title) {}
4
5 // Activar binding
6 a. @EnableConfigurationProperties(AppProps.class)
```

Listing 5: ConfigurationProperties tipado

Idea: configura fuera del código y bindéalo a tipos seguros.

3.5 Datos con JPA e H2 (memoria)

```
1 @Entity class Note { @Id @GeneratedValue Long id; String text; }
2 interface NoteRepo extends JpaRepository<Note, Long> {}
```

Listing 6: Entidad y repositorio

```
1 spring.datasource.url: jdbc:h2:mem:demo;DB_CLOSE_DELAY=-1
2 spring.jpa.hibernate.ddl-auto: update
```

Listing 7: BD en memoria para desarrollo

Idea: con pocas líneas guardas y lees datos reales.

3.6 Validación y manejo de errores

```
1 record CreateNote(@jakarta.validation.constraints.NotBlank String text) {}
2
3 @RestController class NotesCtrl {
4     private final NoteRepo repo;
5     NotesCtrl(NoteRepo repo){ this.repo = repo; }
6     @PostMapping("/notes") Note create(@Valid @RequestBody CreateNote body){
7         var n = new Note(); n.text = body.text(); return repo.save(n);
8     }
9 }
```

Listing 8: Validar entrada y responder 400

Idea: anota DTOs y deja que Spring devuelva 400 automáticamente si no cumplen.

Consejo Senior

Construye siempre así: (1) endpoint mínimo, (2) servicio, (3) propiedades, (4) datos, (5) validación/errores.

4 Ecosistema y proyecto

Qué significa Spring Boot trae “piezas” preconfiguradas (web, datos, seguridad, observabilidad). Arrancas rápido sin pelear con configuración de bajo nivel.

En la práctica Elige *starters* según la necesidad (spring-boot-starter-web, -data-jpa, -security). Usa perfiles (dev/prod) en application-*.yaml.

- **Spring Boot 3.x:** migración a jakarta.*, configuración por convención y *starters*.
- **Build:** Maven/Gradle, Buildpacks para imágenes, perfiles y entornos.
- **Observabilidad:** Actuator, Micrometer, trazas con OTel.
- **Ejecución:** JVM, contenedores, *native images* con GraalVM.

4.1 Estructura mínima

Qué mirar La clase `@SpringBootApplication` y el `application.yml`. Luego, paquetes por **feature**: `web/`, `core/`, `data/`.

```
1 // src/main/java/com/example/App.java
2 package com.example;
3
4 import org.springframework.boot.SpringApplication;
5 import org.springframework.boot.autoconfigure.SpringBootApplication;
6 import org.springframework.context.annotation.Bean;
7 import java.util.concurrent.Executor;
8 import java.util.concurrent.Executors;
9
10 @SpringBootApplication
11 public class App {
12     public static void main(String[] args) { SpringApplication.run(App.class,
13         args); }
14
15     // Executor con virtual threads (Java 21+, Spring Boot 3.2+)
16     @Bean Executor appExecutor() { return Executors.
17         newVirtualThreadPerTaskExecutor(); }
```

Listing 9: Aplicación principal y configuración de beans

```
1 spring:
2   application:
3     name: demo
4   profiles:
5     active: dev
6
7 management:
8   endpoints:
9     web:
10       exposure:
11         include: "*"
12   tracing:
13     enabled: true
14     sampling:
15       probability: 1.0
16   metrics:
17     tags:
18       application: ${spring.application.name}
```

Listing 10: `application.yml` con perfiles y observabilidad

5 Configuración tipada y propiedades

Por qué importa Evita strings sueltos y errores de configuración. Con records y validación fallarás rápido y claro al arrancar.

```
1 // src/main/java/com/example/config/AppProps.java
2 package com.example.config;
3
4 import org.springframework.boot.context.properties.ConfigurationProperties;
5 import org.springframework.validation.annotation.Validated;
6 import jakarta.validation.constraints.NotBlank;
7 import java.time.Duration;
```

```

8
9 @Validated
10 @ConfigurationProperties(prefix = "app")
11 public record AppProps(@NotBlank String title, Duration timeout) {}

```

Listing 11: Configuration Properties con records y validación

```

1 // src/main/java/com/example/config/Config.java
2 package com.example.config;
3
4 import org.springframework.boot.context.properties.
    EnableConfigurationProperties;
5 import org.springframework.context.annotation.Configuration;
6
7 @Configuration
8 @EnableConfigurationProperties(AppProps.class)
9 class Config {}

```

Listing 12: Activación de binding tipado

```

1 app:
2   title: Demo App
3   timeout: 5s

```

Listing 13: Valores de ejemplo

6 Auto-configuración: cómo funciona por debajo

Idea clave Spring Boot activa beans según “si está presente tal clase/propiedad y falta tal bean”. Así te ahorra wiring manual.

En la práctica Si algo no se activa, revisa el *Condition Evaluation Report* o `/actuator/conditions`.

- Spring Boot importa **autoconfiguraciones** listadas en `META-INF/spring/org.springframework.boot.autoconfigure`
- Cada clase `@AutoConfiguration` declara `@ConditionalOnClass`, `OnMissingBean`, `OnProperty`, etc. El *ConditionEvaluationReport* explica por qué se activa o no.
- Orden: `@AutoConfigureBefore/After`. Desactivar con `spring.autoconfigure.exclude`.

```

1 package com.example.starter;
2
3 import org.springframework.boot.autoconfigure.AutoConfiguration;
4 import org.springframework.boot.autoconfigure.condition.ConditionalOnClass;
5 import org.springframework.boot.autoconfigure.condition.
    ConditionalOnMissingBean;
6 import org.springframework.context.annotation.Bean;
7
8 @AutoConfiguration
9 @ConditionalOnClass(name = "com.example.lib.Client")
10 public class ClientAutoConfiguration {
11
12     @Bean
13     @ConditionalOnMissingBean
14     public ClientApi clientApi() { return new ClientApi(); }
15 }

```

Listing 14: AutoConfiguration mínima condicional

```
1 # META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.  
  imports  
2 com.example.starter.ClientAutoConfiguration
```

Listing 15: Registrar la autoconfiguración**Consejo Senior**

Para depurar condiciones: arranca con `-debug` o revisa el *Condition Evaluation Report* en logs. Usa Actuator endpoint `/actuator/conditions`.

7 Web MVC y REST

Cómo diseñar Controladores finos que validan y orquestan; servicios con la lógica; DTOs para entrada/salida; `@ControllerAdvice` para errores uniformes.

```
1 package com.example.web;  
2  
3 import org.springframework.web.bind.annotation.*;  
4 import jakarta.validation.Valid;  
5 import jakarta.validation.constraints.*;  
6 import org.springframework.http.ResponseEntity;  
7  
8 record UserDto(@NotBlank String id, @Email String email) {}  
9  
10 @RestController  
11 @RequestMapping("/api/users")  
12 class UserController {  
13     @GetMapping("/{id}")  
14     ResponseEntity<UserDto> get(@PathVariable String id) {  
15         return ResponseEntity.ok(new UserDto(id, "user@example.com"));  
16     }  
17  
18     @PostMapping  
19     ResponseEntity<UserDto> create(@Valid @RequestBody UserDto body) {  
20         return ResponseEntity.status(201).body(body);  
21     }  
22 }
```

Listing 16: Controlador REST con validación y manejo de errores

```
1 package com.example.web;  
2  
3 import org.springframework.web.bind.annotation.*;  
4 import org.springframework.http.*;  
5 import org.springframework.web.method.annotation.  
  MethodArgumentTypeMismatchException;  
6 import org.springframework.validation.BindException;  
7  
8 @RestControllerAdvice  
9 class GlobalErrors {  
10     @ExceptionHandler({ BindException.class,  
  MethodArgumentTypeMismatchException.class })  
11     ResponseEntity<?> badReq(Exception ex) {  
12         return ResponseEntity.badRequest().body(new ErrorPayload("BAD_REQUEST",  
  ex.getMessage()));  
13     }  
14 }
```

```

15 record ErrorPayload(String code, String message) {}
16 }

```

Listing 17: Advice global para errores

8 Datos: JPA, transacciones y migraciones

Reglas prácticas - Transacciones en servicios. - Evita exponer entidades en la API, usa DTOs. - Controla N+1 con EntityGraph o *fetch join*.

```

1 package com.example.data;
2
3 import jakarta.persistence.*;
4 import org.springframework.data.jpa.repository.*;
5 import org.springframework.data.repository.query.Param;
6 import java.util.*;
7
8 @Entity
9 @Table(name = "users")
10 public class User {
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
12     Long id;
13     @Column(nullable = false, unique = true)
14     String email;
15 }
16
17 interface UserRepo extends JpaRepository<User, Long> {
18     Optional<User> findByEmail(@Param("email") String email);
19 }

```

Listing 18: Entidad JPA y repositorio

```

1 package com.example.data;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.transaction.annotation.Transactional;
5
6 @Service
7 class UserService {
8     private final UserRepo repo;
9     UserService(UserRepo repo) { this.repo = repo; }
10
11     @Transactional
12     public User create(String email) {
13         var u = new User();
14         u.email = email;
15         return repo.save(u);
16     }
17 }

```

Listing 19: Servicio con @Transactional y reglas clave

```

1 spring:
2   datasource:
3     url: jdbc:postgresql://localhost:5432/app
4     username: app
5     password: secret
6   jpa:

```



```

7   hibernate:
8     ddl-auto: validate
9   properties:
10    hibernate:
11      format_sql: true
12      jdbc.batch_size: 50
13  flyway:
14    enabled: true
15    locations: classpath:db/migration

```

Listing 20: Configuración JPA y Flyway**Patrón**

Evita `LazyInitializationException`: diseña límites de transacción en servicios, usa *fetch joins* o *entity graphs*. Controla el N+1 con logs y `hibernate.default_batch_fetch_size`.

9 Seguridad con Spring Security 6

Modelo mental Una cadena de filtros decide quién eres (autenticación) y qué puedes hacer (autorización). Mantén reglas simples y centralizadas.

```

1  package com.example.security;
2
3  import org.springframework.context.annotation.*;
4  import org.springframework.security.config.annotation.web.builders.
    HttpSecurity;
5  import org.springframework.security.web.SecurityFilterChain;
6  import org.springframework.security.config.Customizer;
7
8  @Configuration
9  class SecurityConfig {
10     @Bean
11     SecurityFilterChain http(HttpSecurity http) throws Exception {
12         http
13             .csrf(csrf -> csrf.disable())
14             .cors(Customizer.withDefaults())
15             .authorizeHttpRequests(auth -> auth
16                 .requestMatchers("/actuator/**").permitAll()
17                 .requestMatchers("/api/admin/**").hasRole("ADMIN")
18                 .anyRequest().authenticated())
19             .oauth2ResourceServer(oauth -> oauth.jwt(Customizer.withDefaults()));
20         return http.build();
21     }
22 }

```

Listing 21: SecurityFilterChain, CORS y JWT Resource Server

```

1  spring:
2    security:
3      oauth2:
4        resourceserver:
5          jwt:
6            jwk-set-uri: https://issuer.example.com/.well-known/jwks.json

```

Listing 22: Propiedades JWT (ejemplo con jwk-set-uri)

Consejo Senior

Método seguro: validación a nivel de método con `@EnableMethodSecurity` y anotaciones `@PreAuthorize("hasRole('ADMIN')")`. Externaliza autorizaciones complejas a un servicio.

10 Reactividad y WebFlux

Cuándo usarlo I/O intensivo con muchas conexiones y latencia de redes. Si tu stack es bloqueante (JPA), quédate en MVC.

```
1 package com.example.reactive;
2
3 import org.springframework.web.bind.annotation.*;
4 import reactor.core.publisher.*;
5 import org.springframework.web.reactive.function.client.WebClient;
6
7 @RestController
8 class ReactiveCtrl {
9     private final WebClient http = WebClient.create();
10
11     @GetMapping("/rx") Mono<String> hello() {
12         return Mono.just("ok");
13     }
14
15     @GetMapping("/rx/users") Flux<String> users() {
16         return http.get().uri("https://example.com/users")
17             .retrieve().bodyToFlux(String.class);
18     }
19 }
```

Listing 23: Controlador reactivo y WebClient

Evita esto

No mezcles WebFlux con JPA bloqueante; usa R2DBC o drivers no bloqueantes. Si debes integrar, ejecuta en `boundedElastic` pero entiende el coste.

11 Mensajería, tareas y caché

11.1 Kafka / RabbitMQ

```
1 package com.example.msg;
2
3 import org.springframework.kafka.annotation.KafkaListener;
4 import org.springframework.kafka.core.KafkaTemplate;
5 import org.springframework.stereotype.Service;
6
7 @Service
8 class KafkaSvc {
9     private final KafkaTemplate<String, String> kt;
10     KafkaSvc(KafkaTemplate<String, String> kt) { this.kt = kt; }
11     void publish(String topic, String value) { kt.send(topic, value); }
12
13     @KafkaListener(topics = "events", groupId = "app")
14     void consume(String value) { System.out.println("event=" + value); }
15 }
```

Listing 24: Productor/consumidor Kafka

11.2 Tareas y scheduling

```
1 package com.example.bg;
2
3 import org.springframework.scheduling.annotation.*;
4 import org.springframework.context.annotation.*;
5 import java.util.concurrent.*;
6
7 @Configuration
8 @EnableAsync
9 @EnableScheduling
10 class BgConfig {
11     @Bean Executor taskExecutor() { return Executors.newFixedThreadPool(8); }
12
13     @Async
14     public void work() { /* ... */ }
15
16     @Scheduled(cron = "0 */5 * * * *")
17     public void job() { /* ... */ }
18 }
```

Listing 25: @Async y @Scheduled con pools dedicados

11.3 Caché

```
1 package com.example.cache;
2
3 import org.springframework.cache.annotation.*;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
7 @EnableCaching
8 class CacheCfg {}
9
10 class ProductService {
11     @Cacheable(cacheNames = "products", key = "#id")
12     Product findById(Long id) { /* lookup */ return null; }
13 }
```

Listing 26: Caché con Caffeine/Redis

12 Observabilidad con Actuator y Micrometer

Qué exponer Salud, métricas y trazas. Etiqueta métricas con nombre de aplicación y versión. Envía trazas a tu collector (OTel).

```
1 management:
2   endpoints.web.exposure.include: "*"
3   metrics.export.prometheus.enabled: true
4   otlp:
5     tracing:
6       endpoint: http://otel-collector:4318/v1/traces
```

Listing 27: Actuator y exportación de métricas/trazas

```

1 package com.example.obs;
2
3 import io.micrometer.core.instrument.MeterRegistry;
4 import org.springframework.stereotype.Component;
5
6 @Component
7 class Metrics {
8     Metrics(MeterRegistry registry) {
9         registry.counter("app.startups").increment();
10    }
11 }

```

Listing 28: Métricas personalizadas

13 Testing eficaz

Estrategia mínima - Unit: servicios con mocks. - Slices: @WebMvcTest, @DataJpaTest. - Integración: Testcontainers para BD real. - Smoke: endpoints de Actuator.

```

1 package com.example;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.springframework.boot.test.autoconfigure.web.servlet.
    AutoConfigureMockMvc;
6 import org.springframework.test.web.servlet.MockMvc;
7 import org.springframework.beans.factory.annotation.Autowired;
8
9 import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.get;
10 import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.status;
11
12 @SpringBootTest
13 @AutoConfigureMockMvc
14 class AppTests {
15     @Autowired MockMvc mvc;
16
17     @Test void health() throws Exception {
18         mvc.perform(get("/actuator/health")).andExpect(status().isOk());
19     }
20 }

```

Listing 29: @SpringBootTest + MockMvc

```

1 package com.example;
2
3 import org.junit.jupiter.api.*;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.testcontainers.containers.PostgreSQLContainer;
6 import org.springframework.test.context.DynamicPropertyRegistry;
7 import org.springframework.test.context.DynamicPropertySource;
8
9 @SpringBootTest
10 class PgIT {
11     static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:15")
12         ;
13     static { pg.start(); }

```

```
13
14 @DynamicPropertySource
15 static void props(DynamicPropertyRegistry r) {
16     r.add("spring.datasource.url", pg::getJdbcUrl);
17     r.add("spring.datasource.username", pg::getUsername);
18     r.add("spring.datasource.password", pg::getPassword);
19 }
20 }
```

Listing 30: Testcontainers para PostgreSQL

14 Empaquetado, Docker y Native Image

Ruta práctica Desarrollo con `spring-boot:run`, empaqueta JAR, luego imagen con Buildpacks. Prueba Native si necesitas arranque ultrarrápido y menor memoria.

```
1 # Ejecutar en desarrollo
2 mvn spring-boot:run
3
4 # Empaquetar jar
5 mvn -DskipTests package
6
7 # Imagen con buildpacks
8 mvn spring-boot:build-image -DskipTests
9
10 # Imagen nativa (GraalVM Native Build Tools instalado)
11 mvn -Pnative native:compile
```

Listing 31: Comandos útiles Maven/Gradle y contenedores

Evita esto

AOT/native reduce memoria y latencia de arranque pero limita reflexión y proxys dinámicos. Verifica compatibilidad de librerías (hints) y usa `@RegisterReflectionForBinding` cuando sea necesario.

15 Rendimiento y escalabilidad

Por dónde empezar Mide primero. Ajusta pool de conexiones, usa paginación, evita trabajo en el hilo de petición, añade caché donde tenga sentido.

- Conexiones: HikariCP, tamaño de pool acorde al workload; en reactividad, evita pools bloqueantes.
- GC y memoria: `-Xms/-Xmx`, G1/Generational ZGC según JDK.
- Virtual Threads: alto *throughput* I/O-bound con código imperativo; mide latencias y bloqueo de drivers.
- Warmup y JIT: *tiered compilation*; en nativo no hay JIT, calibra tiempo de arranque vs rendimiento sostenido.

16 Patrones y anti-patrones

Patrón

Patrones: Hexagonal/Ports-Adapters, Domain Services, DTO/assembler, Outbox para consistencia entre BDs y colas, Idempotency keys.

Evita esto

Anti-patrones: entidades anémicas con lógica en controladores, transacciones cruzando límites externos, `@Transactional` en métodos `private`, *God Services*, repositorios chatos que exponen JPA al resto del dominio.

17 Checklist de revisión senior

1. Configuración tipada (`@ConfigurationProperties`) y validada.
2. Límites de transacción claros; sin N+1; migraciones con Flyway/Liquibase.
3. Seguridad centralizada; endpoints expuestos y CORS definidos.
4. Observabilidad: Actuator, métricas y trazas; logs estructurados.
5. Tests con slices y Testcontainers para integrar con dependencias.
6. Dockerfile o Buildpacks; perfiles por entorno; *readiness/liveness* para K8s.

18 Spring Core avanzado (más allá de Spring Boot)

18.1 Contenedor IoC, ciclos de vida y extensibilidad

- **BeanFactory vs ApplicationContext**: este último añade internacionalización, eventos, recursos y autowiring avanzado.
- **Ciclo de vida**: instanciación → inyección → callbacks (`InitializingBean/@PostConstruct`) → post-procesadores → `SmartInitializingSingleton` → destrucción (`DisposableBean/@PreDestroy`).
- **Scopes**: singleton, prototype y web (request/session). Cuidado con inyectar prototype en singleton.
- Extensión: `BeanFactoryPostProcessor`, `BeanDefinitionRegistryPostProcessor`, `BeanPostProcessor`, `FactoryBean`, eventos (`ApplicationEventPublisher/@EventListener`).

```
1 // Post-procesador que aplica un wrapper condicional
2 @Component
3 class TimingPostProcessor implements BeanPostProcessor {
4     @Override public Object postProcessAfterInitialization(Object bean, String
5         name) {
6         if (bean instanceof Service) return ProxyFactory.getProxy(bean.getClass()
7             , new TimingInterceptor(bean));
8         return bean;
9     }
10 }
11
12 // FactoryBean crea un bean complejo bajo un id simple
13 @Component("client")
14 class ClientFactory implements FactoryBean<Client> {
15     public Client getObject() { return new Client("cfg"); }
16     public Class<?> getObjectType() { return Client.class; }
```

```
15 }
```

Listing 32: BeanPostProcessor y FactoryBean

```
1 @Component
2 class Eventing {
3     private final ApplicationEventPublisher pub;
4     Eventing(ApplicationEventPublisher pub) { this.pub = pub; }
5     public void fire(User u) { pub.publishEvent(new UserCreated(u.id())); }
6 }
7
8 record UserCreated(Long id) {}
9
10 @Component
11 class Audits {
12     @EventListener
13     void on(UserCreated e) { /* antes de commit */ }
14
15     @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
16     void onCommitted(UserCreated e) { /* side-effects confiables */ }
17 }
```

Listing 33: Eventos y listeners transaccionales

18.2 AOP en profundidad

- **Proxies** JDK (interfaces) y CGLIB (clases). Problema clásico: *self-invocation* no pasa por el proxy.
- Tipos de advice: @Before, @AfterReturning, @AfterThrowing, @Around. Diseño de pointcuts con AspectJ (execution, within, @annotation, etc.).
- Ordenación con @Order; minimizar *pointcuts* amplios para evitar impacto en rendimiento.

```
1 @Aspect
2 @Component
3 @Order(10)
4 class MetricsAspect {
5     private final MeterRegistry m;
6     MetricsAspect(MeterRegistry m) { this.m = m; }
7
8     @Around("within(@org.springframework.stereotype.Service *)")
9     Object timed(ProceedingJoinPoint pjp) throws Throwable {
10         var sample = Timer.start(m);
11         try { return pjp.proceed(); }
12         finally { sample.stop(m.timer("svc.time", "method", pjp.getSignature().
13             getName())); }
14 }
```

Listing 34: Aspecto de métricas con @Around

18.3 Transacciones avanzadas

- Propagación: REQUIRED (por defecto), REQUIRES_NEW, NESTED, MANDATORY, SUPPORTS, NOT_SUPPORTED, NEVER.
- Aislamiento: READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE, etc. readOnly es una *pista*; con JPA puede optimizar el *flush*.

- Reglas de rollback: por defecto para *unchecked*; usa `rollbackFor` para *checked*.
- Eventos transaccionales: `TransactionSynchronizationManager`; `@TransactionalEventListener` para *after-commit*.

```

1 @Service
2 class OrderService {
3     private final Repo r;
4     OrderService(Repo r) { this.r = r; }
5
6     @Transactional
7     public void place(Order o) { /* REQUIRED */ save(o); charge(o); }
8
9     @Transactional(propagation = Propagation.REQUIRES_NEW)
10    public void audit(Order o) { /* registra en otra tx */ }
11
12    @Transactional(isolation = Isolation.REPEATABLE_READ)
13    public Order findWithLock(Long id) {
14        return r.findByIdForUpdate(id); // SELECT ... FOR UPDATE (pessimistic)
15    }
16 }

```

Listing 35: Propagación y locking

Evita esto

No llares métodos `@Transactional` de la misma instancia (*self-invocation*): el proxy no intercepta. Extrae a otro bean.

19 Spring Data avanzado

- Ciclo de vida JPA: estados (*transient*, *managed*, *detached*, *removed*), *dirty checking* y *flush*.
- **Proyecciones** (interfaces/clases), **Specifications** y **Querydsl**; **EntityGraph** para asociaciones.
- Auditoría (`@CreatedDate`/`@LastModifiedDate`) y `AuditorAware`.
- Repositorios personalizados y `@Query` para DTOs.

```

1 interface UserView { Long getId(); String getEmail(); }
2
3 interface UserRepo extends JpaRepository<User, Long>,
4     JpaSpecificationExecutor<User> {
5     @EntityGraph(attributePaths = {"roles"})
6     Optional<User> findWithRolesById(Long id);
7
8     @Query("select u.id as id, u.email as email from User u where u.email like :q")
9     List<UserView> search(@Param("q") String q);
10 }

```

Listing 36: EntityGraph y proyecciones

```

1 class Specs {
2     static Specification<User> emailContains(String q) {
3         return (root, cq, cb) -> cb.like(root.get("email"), "%" + q + "%");
4     }
5     static Specification<User> hasRole(String role) {
6         return (root, cq, cb) -> cb.isMember(role, root.get("roles"));
7     }
8 }

```



```
8 }

```

Listing 37: Specifications para filtros dinámicos

20 Spring Security avanzado

- Arquitectura: cadena de filtros, `AuthenticationManager`, `AuthenticationProvider`, `SecurityContext`.
- Autorización moderna con `AuthorizationManager`; seguridad a nivel de método con `@EnableMethodSecurity`.
- Múltiples cadenas `HttpSecurity` con `@Order` para separar APIs públicas/privadas.
- **`PermissionEvaluator`** para reglas de dominio; sesiones *stateless* y `SessionCreationPolicy.STATELESS`.

```

1  @Configuration
2  @EnableMethodSecurity
3  class MultiHttp {
4      @Bean @Order(1)
5      SecurityFilterChain api(HttpSecurity http) throws Exception {
6          http.securityMatcher("/api/**")
7              .authorizeHttpRequests(a -> a.anyRequest().authenticated())
8              .oauth2ResourceServer(o -> o.jwt());
9          return http.build();
10     }
11     @Bean @Order(2)
12     SecurityFilterChain web(HttpSecurity http) throws Exception {
13         http.authorizeHttpRequests(a -> a.anyRequest().permitAll());
14         return http.build();
15     }
16 }
17
18 @Component
19 class DomainPermission implements PermissionEvaluator {
20     public boolean hasPermission(Authentication a, Object target, Object perm)
21     {
22         return /* lógica de dominio */ true; }
23     public boolean hasPermission(Authentication a, Serializable id, String type
24         , Object perm) { return true; }
25 }

```

Listing 38: Múltiples `HttpSecurity` y `PermissionEvaluator`

21 MVC/WebFlux: resolvers, handlers y funcional

21.1 Spring MVC

- **`ArgumentResolvers`**: inyectan parámetros en controladores; **`ReturnValueHandlers`**: serializan respuestas.
- Negociación de contenido, validación y `HttpMessageConverters` (Jackson, etc.).

```

1  @Configuration
2  class MvcCfg implements WebMvcConfigurer {
3      @Override public void addArgumentResolvers(List<
4          HandlerMethodArgumentResolver> resolvers) {
5          resolvers.add(new UserIdResolver());
6      }
7  }

```

```
7
8 class UserId {}
9 class UserIdResolver implements HandlerMethodArgumentResolver {
10     public boolean supportsParameter(MethodParameter p) { return p.
        getParameterType().equals(UserId.class); }
11     public Object resolveArgument(MethodParameter p, ModelAndViewContainer m,
        NativeWebRequest w, WebDataBinderFactory b) {
12         return new UserId();
13     }
14 }
```

Listing 39: ArgumentResolver personalizado

21.2 Spring WebFlux

- Modelo *reactivo* basado en **Reactor**: *event loop*, *backpressure* y *Schedulers*.
- Endpoints funcionales con *RouterFunction/HandlerFunction*.

```
1 @Configuration
2 class Routes {
3     @Bean RouterFunction<ServerResponse> router() {
4         return RouterFunctions.route()
5             .GET("/hello", req -> ServerResponse.ok().bodyValue("hi"))
6             .build();
7     }
8 }
```

Listing 40: Router funcional en WebFlux

22 Integración, Batch y resiliencia

22.1 Spring Integration

- Colas, canales, *gateways*, transformadores y *Enterprise Integration Patterns*.

22.2 Spring Batch

- **Jobs/Steps**: *chunk-oriented* vs *tasklet*. Reintentables, idempotencia y control de estado.

22.3 Resilience4j y Spring Cloud

- Circuit Breaker, Retry, RateLimiter, Bulkhead. Configuración por instancia.
- Spring Cloud: Config, Gateway, OpenFeign, LoadBalancer; trazas con Micrometer Tracing/OTel.

```
1 resilience4j.circuitbreaker:
2   instances:
3     externalApi:
4       slidingWindowSize: 20
5       failureRateThreshold: 50
```

Listing 41: Resilience4j por instancia

23 Configuración avanzada y PropertySources

- Orden de resolución: args de línea de comandos → variables de entorno → archivos `application-*.yaml` → `@PropertySource`.
- `Environment` y `PropertySources`; perfiles y `@Profile`, `@Conditional` personalizadas.
- Cifrado de propiedades (JCE/HashiCorp Vault/SOPS) y secretos externos.

```
1 class OnLinux implements Condition {
2     public boolean matches(ConditionContext c, AnnotatedTypeMetadata m) {
3         return c.getEnvironment().getProperty("os.name", "").toLowerCase().
4             contains("linux");
5     }
6 }
7 @Configuration
8 @Conditional(OnLinux.class)
9 class LinuxOnly {}
```

Listing 42: Condition personalizada

24 Testing avanzado

- **Slices:** `@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, `@RestClientTest`.
- `@MockBean` para sustituir dependencias; **context caching** para acelerar.
- `WireMock`/`Testcontainers` para dependencias externas (HTTP/Kafka/Redis).

```
1 @WebMvcTest(UserController.class)
2 class UserControllerTest {
3     @Autowired MockMvc mvc;
4     @MockBean UserService svc;
5
6     @Test void ok() throws Exception {
7         when(svc.get(anyString())).thenReturn(new UserDto("1", "a@b.com"));
8         mvc.perform(get("/api/users/1")).andExpect(status().isOk());
9     }
10 }
```

Listing 43: `@WebMvcTest` con servicio simulado

25 Plan de estudio Senior Spring (6 semanas)

1. Core IoC, ciclo de vida, eventos, AOP básico; labs con `BeanPostProcessor` y `FactoryBean`.
2. Transacciones avanzadas, locking, specs JPA y proyecciones; pruebas con `@DataJpaTest` y `Testcontainers`.
3. Security avanzado (multi-HttpSecurity, método), MVC internals (resolvers/handlers).
4. WebFlux: router funcional, backpressure, integración con R2DBC.
5. Integración/Batch/Resilience4j; observabilidad profunda con métricas y trazas.
6. Cloud: Config, Gateway, OpenFeign; despliegue en contenedores y rendimiento.

26 API y serialización avanzada

26.1 Validación avanzada y method validation

```
1 import jakarta.validation.constraints.*;
2 import org.springframework.validation.annotation.Validated;
3
4 interface Create {}
5 interface Update {}
6
7 record Product(@NotNull(groups=Update.class) Long id,
8               @NotBlank(groups={Create.class, Update.class}) String name) {}
9
10 @Validated
11 class ProductService {
12     public @NotBlank String normalize(@NotBlank String name) { return name.trim
13                                     (); }
14 }
```

Listing 44: Groups y validación a nivel de método

26.2 Jackson: configuración global y JavaTime

```
1 @Configuration
2 class JacksonCfg {
3     @Bean com.fasterxml.jackson.databind.Module javaTime() {
4         return new com.fasterxml.jackson.datatype.jsr310.JavaTimeModule();
5     }
6 }
```

Listing 45: ObjectMapper personalizado

26.3 HTTP caching, ETag y Problem+JSON

```
1 @GetMapping("/items/{id}")
2 ResponseEntity<ItemDto> get(@PathVariable Long id, WebRequest req) {
3     var item = svc.get(id);
4     var etag = '"' + item.version() + '"';
5     if (req.checkNotModified(etag)) return null;
6     return ResponseEntity.ok().eTag(etag).cacheControl(CacheControl.maxAge(
7         Duration.ofMinutes(5))).body(item);
8 }
```

Listing 46: ETag condicional y Cache-Control

Consejo Senior

Estandariza errores con application/problem+json; usa un `@ControllerAdvice` que serialice código, título y detalle de la incidencia.

27 JPA avanzada: concurrencia, caché y paginación

27.1 Optimistic/Pessimistic locking

```
1 @Entity
2 class Account {
3     @Id Long id;
4     @Version Long version;
5     BigDecimal balance;
6 }
7
8 interface AccountRepo extends JpaRepository<Account, Long> {
9     @Lock(LockModeType.PESSIMISTIC_WRITE)
10    @Query("select a from Account a where a.id=:id")
11    Account findForUpdate(@Param("id") Long id);
12 }
```

Listing 47: @Version y bloqueo pesimista

27.2 Caché de segundo nivel y batch

```
1 spring:
2   jpa:
3     properties:
4       hibernate.cache.use_second_level_cache: true
5       hibernate.cache.region.factory_class: org.hibernate.cache.jcache.
6         JCacheRegionFactory
7       hibernate.javax.cache.provider: org.ehcache.jsr107.
8         EhcacheCachingProvider
9       hibernate.default_batch_fetch_size: 50
```

Listing 48: Ehcache/Hibernate y batch

27.3 Paginación correcta

```
1 Page<User> page = repo.findAll(PageRequest.of(0, 50, Sort.by("id")));
2 Slice<User> slice = repo.findAllByActiveTrue(PageRequest.of(0, 50));
```

Listing 49: Page vs Slice y orden estable

28 Mapeo eficiente con MapStruct

```
1 @Mapper(componentModel = "spring")
2 interface UserMapper {
3     UserDto toDto(User e);
4     @Mapping(target="id", ignore=true)
5     User toEntity(CreateUserDto dto);
6 }
```

Listing 50: Mapper con @Mapper(componentModel="spring")

29 OAuth2/OIDC avanzado

```
1 @Configuration
2 class OAuthCfg {
3     @Bean SecurityFilterChain login(HttpSecurity http) throws Exception {
4         http.securityMatcher("/ui/**")
5             .authorizeHttpRequests(a -> a.anyRequest().authenticated())
6     }
```

```
6     .oauth2Login(Customizer.withDefaults());
7     return http.build();
8 }
9 @Bean SecurityFilterChain api(HttpSecurity http) throws Exception {
10     http.securityMatcher("/api/**")
11         .authorizeHttpRequests(a -> a.anyRequest().authenticated())
12         .oauth2ResourceServer(o -> o.jwt());
13     return http.build();
14 }
15 }
```

Listing 51: Client OIDC (login) y Resource Server

30 Despliegue cloud-native y Kubernetes

```
1 apiVersion: apps/v1
2 kind: Deployment
3 spec:
4   template:
5     spec:
6       containers:
7       - name: app
8         image: app:latest
9         readinessProbe:
10           httpGet: { path: /actuator/health/readiness, port: 8080 }
11         livenessProbe:
12           httpGet: { path: /actuator/health/liveness, port: 8080 }
13         resources:
14           requests: { cpu: "200m", memory: "256Mi" }
15           limits: { cpu: "1", memory: "512Mi" }
```

Listing 52: Probes y recursos

31 OpenAPI y documentación

```
1 // build: añade dependency org.springdoc:springdoc-openapi-starter-webmvc-ui
2 @Configuration
3 class OpenApiCfg {}
4 // UI: /swagger-ui.html Spec: /v3/api-docs
```

Listing 53: springdoc-openapi configuración mínima

32 Logging estructurado y correlación

```
1 import org.slf4j.MDC;
2
3 try (var ignored = MDC.putCloseable("corrId", UUID.randomUUID().toString()))
4 {
5     log.info("procesando");
6 }
```

Listing 54: MDC para correlation id

```
1 logging:
2   pattern:
3     console: "%d{ISO8601} %level %logger - %X{corrId} %msg%n"
```

Listing 55: Logback JSON (ejemplo)

33 Build: Gradle y Jib

```
1 ./gradlew bootRun
2 ./gradlew bootJar
3 ./gradlew jibDockerBuild # com.google.cloud.tools:jib-spring-boot-extension
```

Listing 56: Gradle y Jib

34 R2DBC: datos reactivos

```
1 interface UserRxRepo extends org.springframework.data.r2dbc.repository.
   R2dbcRepository<User, Long> {}
2
3 @Service
4 class RxSvc {
5   private final org.springframework.transaction.reactive.
     TransactionalOperator tx;
6   private final UserRxRepo repo;
7   Mono<User> create(User u) {
8     return tx.execute(status -> repo.save(u));
9   }
10 }
```

Listing 57: Repositorio reactivo y tx

35 Patrón Outbox para consistencia

```
1 @Entity class OutboxEvent { @Id @GeneratedValue Long id; String type; String
   payload; Instant createdAt; }
2
3 @Service
4 class OutboxPublisher {
5   @Transactional
6   public void createAndPublish(Order order) {
7     // 1) Persistir dominio
8     // 2) Persistir evento outbox en la misma TX
9   }
10 }
```

Listing 58: Entidad Outbox y publicador

A Comandos CLI y propiedades clave

- `mvn spring-boot:run` `./mvnw` disponible en proyectos generados.
- `mvn -DskipTests package` empaqueta JAR.

- `mvn spring-boot:build-image` imagen OCI con Buildpacks.
- Perfiles: `-spring.profiles.active=prod`. Variables: `SPRING_APPLICATION_NAME`.

B Cadena de filtros y flujo MVC (internals)

- Servlet Container → FilterChain → DispatcherServlet → HandlerMapping → HandlerAdapter → Controller/Handler → ReturnValueHandlers → ViewResolver.
- Transacciones y proxies: `@Transactional` crea *proxies* (JDK/CGLIB); fuga si se llama internamente al método desde la misma instancia.
- Post-procesadores: `BeanFactoryPostProcessor` y `BeanPostProcessor` permiten modificar definiciones/instancias en el ciclo de vida.

C Certificación Java SE 17 Developer (OCP) — Guía práctica

Esta guía resume los objetivos clave de OCP Java 17 (1Z0-829) y añade ejemplos y ejercicios.

C.1 Temario esencial

- Lenguaje: tipos, operadores, control de flujo, alcance, `var`, `record`, `sealed`, `switch expression`.
- POO: herencia, interfaces, clases anidadas, inmutabilidad, `equals/hashCode/compareTo`.
- Genéricos y colecciones: listas, conjuntos, mapas, comparadores, utilidades de `Collections`.
- Streams y funcional: pipelines, `Optional`, `Collectors`.
- Concurrencia: hilos, `ExecutorService`, colecciones concurrentes, `CompletableFuture`.
- Excepciones: jerarquía, `try-with-resources`, `multi-catch`.
- I/O y NIO.2: `Path`, `Files`, `WatchService`.
- Fecha/hora y `i18n`: `java.time`, `Locale`, `ResourceBundle`.
- Módulos (JPMS): `module-info.java`.
- JVM y herramientas: empaquetado, JAR, JFR/JCMD básico.

C.2 Novedades relevantes y sintaxis moderna

```
1 public record Point(int x, int y) {}
2
3 static String typeOf(Object o) {
4     if (o instanceof String s) return "str:" + s.length();
5     if (o instanceof Number n) return "num:" + n.intValue();
6     return "other";
7 }
8
9 static int dayToNum(String d) {
10    return switch (d) {
11        case "MON", "TUE", "WED", "THU", "FRI" -> 1;
12        case "SAT", "SUN" -> 2;
13        default -> throw new IllegalArgumentException();
14    };
15 }
```

Listing 59: Record, pattern matching y switch expression


```
1 public sealed interface Shape permits Circle, Rect {}
2 public final class Circle implements Shape {}
3 public non-sealed class Rect implements Shape {}
```

Listing 60: Clases selladas (sealed) y jerarquías cerradas

C.3 POO sólida

```
1 public final class User implements Comparable<User> {
2     private final String id; private final String email;
3     public User(String id, String email) { this.id = id; this.email = email; }
4     @Override public boolean equals(Object o) {
5         if (this == o) return true; if (!(o instanceof User u)) return false;
6         return id.equals(u.id);
7     }
8     @Override public int hashCode() { return id.hashCode(); }
9     @Override public int compareTo(User o) { return this.id.compareTo(o.id); }
10 }
```

Listing 61: equals/hashCode/compareTo correctos

C.4 Genéricos y colecciones

```
1 static <T> void copy(List<? extends T> src, List<? super T> dst) {
2     for (T t : src) dst.add(t);
3 }
```

Listing 62: PECS: Producer Extends, Consumer Super

```
1 var m = new java.util.HashMap<String,Integer>();
2 m.merge("a",1,Integer::sum);
3 m.merge("a",1,Integer::sum); // {a=2}
4 var sorted = m.entrySet().stream()
5     .sorted(java.util.Map.Entry.<String,Integer>comparingByValue().reversed())
6     .toList();
```

Listing 63: Operaciones de Map y ordenación

C.5 Streams, Optional y Collectors

```
1 var list = java.util.List.of(1,2,3,4,5);
2 var evensSquared = list.stream()
3     .filter(i -> i % 2 == 0)
4     .map(i -> i * i)
5     .collect(java.util.stream.Collectors.toUnmodifiableList());
```

Listing 64: Pipeline típico y colecciones inmutables

```
1 var byParity = list.stream().collect(java.util.stream.Collectors.
    partitioningBy(i -> i % 2 == 0));
2 var stats = list.stream().collect(java.util.stream.Collectors.summarizingInt(
    Integer::intValue));
```

Listing 65: Collectors avanzados

C.6 Concurrencia y paralelismo

```
1 var pool = java.util.concurrent.Executors.newFixedThreadPool(4);
2 try {
3     var f = java.util.concurrent.CompletableFuture.supplyAsync(() -> 40 + 2,
4         pool)
5         .thenApply(x -> x * 2)
6         .orTimeout(1, java.util.concurrent.TimeUnit.SECONDS);
7     System.out.println(f.join());
8 } finally { pool.shutdown(); }
```

Listing 66: ExecutorService y CompletableFuture

C.7 Excepciones y TWR

```
1 try (var in = java.nio.file.Files.newBufferedReader(java.nio.file.Path.of("
2     data.txt"))) {
3     System.out.println(in.readLine());
4 } catch (java.io.IOException e) {
5     e.printStackTrace();
6 }
```

Listing 67: Try-with-resources y supresión

C.8 I/O, NIO.2 e i18n

```
1 var path = java.nio.file.Path.of("/tmp");
2 try (var s = java.nio.file.Files.walk(path)) {
3     s.filter(java.nio.file.Files::isRegularFile).forEach(System.out::println);
4 }
5 var ws = java.nio.file.FileSystems.getDefault().newWatchService();
6 path.register(ws, java.nio.file.StandardWatchEventKinds.ENTRY_CREATE);
```

Listing 68: Path/Files y WatchService

```
1 var now = java.time.ZonedDateTime.now();
2 var fmt = java.time.format.DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm z",
3     java.util.Locale.US);
4 System.out.println(now.format(fmt));
5 var b = java.util.ResourceBundle.getBundle("messages", new java.util.Locale("
6     es", "ES"));
7 System.out.println(b.getString("hello"));
```

Listing 69: java.time y ResourceBundle

C.9 JPMS y herramientas

```
1 module com.example.app {
2     requires java.logging;
3     exports com.example.api;
4 }
```

Listing 70: module-info.java mínimo

C.10 Trampas típicas y ejercicios

```
1 Integer a = 128, b = 128; System.out.println(a == b); // false
2 Integer x = 127, y = 127; System.out.println(x == y); // true
```

Listing 71: Autoboxing y cache de Integer

- Identifica salida de código: orden de inicialización, sobrecarga vs sobreescritura.
- Corrige genéricos: comodines y límites múltiples; PECS.
- Completa TWR, Files.walk y Collectors.groupingBy.

C.11 Plan de estudio (8 semanas)

1. Sintaxis base y POO.
2. Genéricos y colecciones.
3. Streams/Optional.
4. Excepciones, I/O y NIO.2.
5. Concurrencia: hilos, locks, ExecutorService, CompletableFuture.
6. Fecha/hora e i18n; módulos JPMS.
7. Simulacros de examen y repasos.
8. JFR/JCMD básico y último simulacro.

Consejo Senior

Practica con bancos de preguntas y escribe micro-programas que demuestren cada objetivo. Prioriza la lectura de preguntas con trampas de tipos y promoción numérica.